PROJECT-BASED LEARNING

PHASE-II EVALUATION

FoodRescue

Save Food. Serve People.

Team ID: *DSCPP-III-2026-T320*

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: *DSCPP-III-2026-T320*
Semester: 3rd
Section: CYBER,DS1,DS2
Domain: Food Waste Management

Team Members

1. *Akshat Raj Singh – 2510380179*
2. *Hardik Raturi – 2510380119*
3. *Aadi Aarav Gupta – 2510388268*

Mentor

Ms. Meenakshi Maindola

Interactions completed: _____

Brief Discussion: Introduce the team, project domain, mentor, and responsibilities of the team members.

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed a C++ food-rescue system connecting donors, recipients and volunteers. Achieved a modular 25-file codebase, a 16-option CLI and a web dashboard over the same managers.

Feedback Received

- Users were handled together; separate them cleanly.
- Strengthen DSA usage beyond arrays and vectors.
- Add expiry handling and real location distance.

Changes Incorporated

- Split into `Donor`, `Recipient`, `Volunteer` under `User`.
- Added linked list, heaps, hash table, graph, FIFO queues.
- Expiry + shelf-life limits; city graph + Dijkstra with map-validated addresses.(future)

Brief Discussion: Explain the major feedback received during Phase-I and clearly demonstrate how it has been incorporated.

Problem Statement & Finalized Objectives

Problem Statement

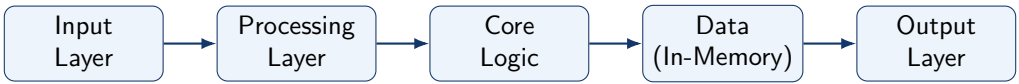
Large quantities of edible food are discarded daily while NGOs and shelters struggle with irregular supply. There is no simple coordinated system for donors to list surplus food, for recipients to raise requests, and for volunteers to deliver it before it expires.

Finalized Objectives

1. Manage donors, recipients, volunteers with fast ID lookup (binary search; hash table).
2. List donations with quantity, type, veg / allergen and shelf-life-checked expiry.
3. Auto-match requests: hard filters, then scored ranking, with a reason per match.
4. Track deliveries with FIFO assignment; every mutation undoable / redoable.

Brief Discussion: Avoid repeating the complete Phase-I presentation. Focus on the finalized problem, scope and measurable objectives.

Updated System Architecture / Workflow



Major Components

- Users (vectors + binary search)
- Food (linked list + expiry heap)
- Requests (max-heap, aging priority)
- Deliveries (FIFO queue)
- Graph + Dijkstra (distances)
- API + HTTP server (dashboard)

Changes Since Phase-I

- Zero headers; 25 `.cpp`, one build.
- CLI + web dashboard (port 8080).
- Scored matching, Dijkstra distance.
- Validation in `Api.cpp` only.

Brief Discussion: Present the updated architecture and explain how individual modules communicate with each other.

Module Design & Functional Requirements

Major Modules

1. User Management (`UserManager`)
2. Food Management (`FoodManager`)
3. Request Management (`RequestManager`)
4. Delivery Management (`DeliveryManager`)
5. Matching Engine (`MatchingEngine`)
6. Graph / Locations (`Graph`)
7. Web + Support (`Api`, `Auth`, `HttpServer`)

Functional Requirements

- Auto IDs: users 1+, food 101+, requests 201+.
- Expire donations; shelf life 2 / 7 / 365 days.
- Create / cancel prioritized requests (with aging).
- Auto-match with a reason; addresses must be map locations.
- Deliveries, FIFO assignment; every mutation undoable.

Module Dependency

`FoodRescueApi` owns the managers, `Graph` and undo/redo stacks; the CLI uses the same managers.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++17 (g++, `-Wall -Wextra`)
- Database: None — in-memory only, by design
- Server: Embedded C++ HTTP (POSIX, port 8080)
- Frontend: Vanilla-JS (`web/index.html`)
- Tools: VS Code; Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Phased Development (Phase 0, 1, 2 done)
4. Integration (CLI + Web, same managers)
5. Testing & Validation (`--test`)

Brief Discussion: Explain why the selected technologies are appropriate and how the team is following the development methodology.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Linked list, FIFO queues, heaps, hash table (djb2), graph + Dijkstra, circular buffer, sorts, binary search	`FoodManager`, `Graph`, `HashTable`
OOPs with C++	Encapsulation, inheritance, polymorphism (virtual functions), overloading, templates, composition	`User`, `Donor`, `Recipient`, `Volunteer`
Operating Systems	POSIX socket HTTP server; epoch time for expiry and priority aging	`HttpServer`, `Util`
DBMS	Not applicable — in-memory management instead of a DBMS, by design	All managers

Important: Mention only those concepts that have been **actually implemented** in the project.

Brief Discussion: Students should explain where and how each course concept is technically implemented.

Development & Implementation Progress

Module	Status	Work Completed
User Management	Completed	`Donor` / `Recipient` / `Volunteer`; binary-search lookup; free-volunteer FIFO queue
Food Management	Completed	Linked list; veg / allergen; shelf-life; merge sort + expiry min-heap
Request Management	Completed	Max-heap, effective priority; counting sort 1–5
Matching Engine	Completed	7 hard filters; score 0.45/0.35/0.20; Dijkstra once; reason per match
Graph, Web & Undo	Completed	Graph + Dijkstra; API + dashboard; undo / redo; demo data
Delivery Management	Core Done	Lifecycle `Pending`–`Delivered`, FIFO; OTP + fail: Phase-III

Overall Project Completion: 85%

Brief Discussion: Separate completed, in-progress and pending work.

Working Prototype / Demonstration

Implemented Features

- 16-option CLI menu (`--cli`) — full workflow
- Web dashboard, port 8080 (admin / admin123)
- Donor → food → request → match → delivery
- Scored matching + reasons (Dijkstra, veg / allergen)
- FIFO assignment; undo / redo; demo data

Prototype / Output

Insert screenshot / output / prototype evidence here

Brief Discussion: Demonstrate the actual working implementation. Students should preferably provide a live demonstration in addition to screenshots wherever applicable.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Expiry & shelf life	`Expired` → never matched; shelf-life limit enforced	As expected	Pass
Priority matching	Highest effective priority first (aging)	As expected	Pass
Location & route	Non-map rejected; route: 7 km via Old Town	As expected	Pass
Scored matching	Filters first; best score wins, reason returned	As expected	Pass

Results / Validation

Compiled with `g++ -std=c++17 -Wall -Wextra`: zero warnings. Self-tests (`--test`): **51/51 pass**; IDs verified via CLI + API.

Brief Discussion: Results are supported by actual testing.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: *Repository URL*
- README.md: *Updated*
- Contributors: *All Members Added*
- Repository Status: *Updated*
- Branches: *If Applicable*

Development Activity

- 25 `.cpp` files + `web/index.html`, module-wise
- Meaningful commit messages per module
- Individual commits visible per member
- Latest build uploaded; 51/51 tests pass
- README with build / run steps

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Brief Discussion: Show repository activity, README, contributor list and commit history during the presentation.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Kapish Singh	Module Lead	System design; manager classes; matching engine; integration testing	*XX*
Member 2	*Role*	*Specific technical work completed*	*XX*
Member 3	*Role*	*Specific technical work completed*	*XX*
Member 4	*Role*	*Specific technical work completed*	*XX*

Individual Evaluation

Every member should be prepared to explain and demonstrate their own technical contribution to the project.

Brief Discussion: Contributions should be specific and verifiable. Avoid generic statements such as "helped in coding" or "worked on documentation".

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- 25 `.cpp` files, zero headers.
- Removing requests from the max-heap; re-ranking on aging.
- Open bugs: `%zz` crash, Match All order, double-booking.

Solutions / Learning

- `main.cpp` includes all `.cpp` in fixed order.
- Heap remove-by-ID + heapify; rebuilt in $O(n)$.
- `recordUndo(...)` after every mutation; IDs keep vectors sorted.

Phase-III Roadmap

1. Fix the three open bugs on the left
2. Timestamps + scored picker (Bike 10 / Car 60 / Van 200)
3. Route + ETA; pickup window in API and UI
4. OTP + fail flow (`Failed`; food back to pool)
5. Knapsack planning (`planBatch`); later phases

Expected Outcomes & References

Expected Final Outcomes

- Complete system: 16-option CLI + web dashboard
- DSA in a real app: linked list, queues, heaps, hash table, graph + Dijkstra
- OOP: inheritance, polymorphism, templates, composition
- Scored matching: shelf-life, veg / allergen, undo / redo
- Ready for Phase-III; report + docs

References

1. Stroustrup, B. — *The C++ Programming Language*
2. `cppreference.com` — C++ documentation
3. Cormen et al. — *Introduction to Algorithms*
4. Graphic Era PBL guidelines and rubric

Thank You
Questions & Suggestions

Brief Discussion: Conclude with the tangible final outcome expected from the project and cite the major technical/research resources used.